REQUESTS

Architectural Manual & Distributed Specification

CHAPTER 1: AUTHENTICATION AND AUTHORIZATION

Overview

The authentication and authorization module is a critical component of the requests library, responsible for handling various authentication schemes and validating user credentials. The module is comprised of multiple classes and functions, each designed to handle specific authentication protocols.

Symbols

Symbol	Description
`_basic_auth_str`	A function that generates a basic authentication string from a username and password.
`AuthBase`	A base class for authentication handlers, providing a common interface for all authentication schemes.
`HTTPBasicAuth`	A class that handles HTTP Basic Authentication, using a username and password to authenticate requests.
`HTTPProxyAuth`	A class that handles HTTP Proxy Authentication, using a username and password to authenticate proxy requests.
`HTTPDigestAuth`	A class that handles HTTP Digest Authentication, using a username and password to authenticate requests with a challenge-response mechanism.

Authentication Handlers

The authentication handlers are responsible for generating the authentication headers and validating user credentials. Each handler implements the `__call__` method, which is used to generate the authentication headers.

```
class AuthBase:
    def __call__(self, request):
        raise NotImplementedError

class HTTPBasicAuth(AuthBase):
    def __init__(self, username, password):
        self.username = username
        self.password = password

    def __call__(self, request):
        auth_str = _basic_auth_str(self.username, self.password)
```

```
        request.headers['Authorization'] = f'Basic {auth_str}'
        return request
```

Authentication Schemes

The authentication schemes are responsible for generating the authentication headers and validating user credentials. Each scheme implements the `authenticate` method, which is used to validate user credentials.

```
class HTTPDigestAuth(AuthBase):
    def __init__(self, username, password):
        self.username = username
        self.password = password

    def authenticate(self, response):
        # Generate the authentication headers
        auth_str = self._generate_auth_str(response)
        response.request.headers['Authorization'] = f'Digest {auth_str}'
        return response
```

Example Usage

```
import requests
from requests.auth import HTTPBasicAuth

# Create an instance of the HTTPBasicAuth class
auth = HTTPBasicAuth('username', 'password')

# Use the authentication handler to authenticate a request
response = requests.get('https://example.com', auth=auth)
```

CHAPTER 2: UTILITIES AND HELPERS

Overview

The Utilities and Helpers module provides various utility functions and classes used throughout the requests library. These utilities help with tasks such as authentication, encoding, and network operations.

Symbols

src/requests/utils.py

Symbol	Description
`dict_to_sequence`	Converts a dictionary to a sequence of key-value pairs.
`super_len`	Returns the length of an object, handling cases where `len()` is not available.
`get_netrc_auth`	Retrieves authentication credentials from a .netrc file.
`guess_filename`	Attempts to guess the filename of a URL.
`extract_zipped_paths`	Extracts the paths from a zip file.
`atomic_open`	Opens a file in atomic write mode.
`from_key_val_list`	Converts a list of key-value pairs to a dictionary.
`to_key_val_list`	Converts a dictionary to a list of key-value pairs.
`parse_list_header`	Parses a list header into individual values.
`parse_dict_header`	Parses a dictionary header into individual key-value pairs.
`unquote_header_value`	Unquotes a header value.
`dict_from_cookiejar`	Converts a cookie jar to a dictionary.
`add_dict_to_cookiejar`	Adds a dictionary to a cookie jar.
`get_encodings_from_content`	Retrieves the encodings from a content type header.
`_parse_content_type_header`	Parses a content type header into individual values.
`get_encoding_from_headers`	Retrieves the encoding from a headers dictionary.
`stream_decode_response_unicode`	Decodes a response stream to Unicode.
`iter_slices`	Iterates over a string in slices.
`get_unicode_from_response`	Retrieves the Unicode content from a response.
`unquote_unreserved`	Unquotes unreserved characters in a string.

Symbol	Description
`requote_uri`	Requotes a URI.
`address_in_network`	Checks if an address is in a network.
`dotted_netmask`	Converts a netmask to a dotted representation.
`is_ipv4_address`	Checks if an address is an IPv4 address.
`is_valid_cidr`	Checks if a CIDR is valid.
`set_environ`	Sets environment variables.
`should_bypass_proxies`	Checks if a request should bypass proxies.
`get_environ_proxies`	Retrieves proxy settings from environment variables.
`select_proxy`	Selects a proxy from a list of proxies.
`resolve_proxies`	Resolves proxies from a dictionary.
`default_user_agent`	Returns the default user agent string.
`default_headers`	Returns the default headers dictionary.
`parse_header_links`	Parses header links into individual values.
`guess_json_utf`	Attempts to guess the UTF encoding of a JSON response.
`prepend_scheme_if_needed`	Prepends a scheme to a URL if necessary.
`get_auth_from_url`	Retrieves authentication credentials from a URL.
`check_header_validity`	Checks the validity of a header.
`_validate_header_part`	Validates a header part.
`urldefragauth`	Removes authentication credentials from a URL.
`rewind_body`	Rewinds a response body.

src/requests/_internal_utils.py

Symbol	Description
`to_native_string`	Converts a string to a native string.
`unicode_is_ascii`	Checks if a Unicode string is ASCII.

tests/test_utils.py

Symbol	Description
`TestSuperLen`	Tests the `super_len` function.

Symbol	Description
`TestGetNetrcAuth`	Tests the `get_netrc_auth` function.
`TestToKeyValList`	Tests the `to_key_val_list` function.
`TestUnquoteHeaderValue`	Tests the `unquote_header_value` function.
`TestGetEnvironProxies`	Tests the `get_environ_proxies` function.
`TestIsIPv4Address`	Tests the `is_ipv4_address` function.
`TestIsValidCIDR`	Tests the `is_valid_cidr` function.
`TestAddressInNetwork`	Tests the `address_in_network` function.
`TestGuessFilename`	Tests the `guess_filename` function.
`TestExtractZippedPaths`	Tests the `extract_zipped_paths` function.
`TestContentEncodingDetection`	Tests content encoding detection.
`TestGuessJSONUTF`	Tests the `guess_json_utf` function.
`test_get_auth_from_url`	Tests the `get_auth_from_url` function.
`test_requote_uri_with_unquoted_percents`	Tests the `requote_uri` function with unquoted percents.
`test_unquote_unreserved`	Tests the `unquote_unreserved` function.
`test_dotted_netmask`	Tests the `dotted_netmask` function.
`test_select_proxies`	Tests the `select_proxy` function.

Symbol	Description
`test_parse_dict_header`	Tests the `parse_dict_header` function.
`test__parse_content_type_header`	Tests the `_parse_content_type_header` function.
`test_get_encoding_from_headers`	Tests the `get_encoding_from_headers` function.
`test_iter_slices`	Tests the `iter_slices` function.
`test_parse_header_links`	Tests the `parse_header_links` function.
`test_prepend_scheme_if_needed`	Tests the `prepend_scheme_if_needed` function.
`test_to_native_string`	Tests the `to_native_string` function.
`test_urldefragauth`	Tests the `urldefragauth` function.
`test_should_bypass_proxies`	Tests the `should_bypass_proxies` function.
`test_should_bypass_proxies_pass_only_hostname`	Tests the `should_bypass_proxies` function with only hostname.
`test_add_dict_to_cookiejar`	Tests the `add_dict_to_cookiejar` function.
`test_unicode_is_ascii`	Tests the `unicode_is_ascii` function.
`test_should_bypass_proxies_no_proxy`	Tests the `should_bypass_proxies` function with no proxy.
`test_should_bypass_proxies_win_registry`	Tests the `should_bypass_proxies` function with Windows registry.

Symbol	Description
`test_should_bypass_proxies_win_registry_bad_values`	Tests the `should_bypass_proxies` function with Windows registry bad values.
`test_set_environ`	Tests the `set_environ` function.
`test_set_environ_raises_exception`	Tests the `set_environ` function raises exception.
`test_should_bypass_proxies_win_registry_ProxyOverride_value`	Tests the `should_bypass_proxies` function with Windows registry ProxyOverride value.

CHAPTER 3: SESSION MANAGEMENT

Overview

The session management module is responsible for managing the lifecycle of a request session. It provides functionality for creating, merging, and redirecting sessions.

Symbols

The following symbols are used in the session management module:

Symbol	Description	Defined in
`merge_setting`	Merges two settings dictionaries into one.	`src/requests/sessions.py`
`merge_hooks`	Merges two hooks dictionaries into one.	`src/requests/sessions.py`
`SessionRedirectMixin`	A mixin class for session redirecting.	`src/requests/sessions.py`
`Session`	A class representing a request session.	`src/requests/sessions.py`
`session`	A function that creates a new session.	`src/requests/sessions.py`

Session Class

The `Session` class represents a request session. It has the following attributes:

Attribute	Description
`headers`	A dictionary of headers for the session.
`cookies`	A dictionary of cookies for the session.
`auth`	An authentication object for the session.
`proxies`	A dictionary of proxies for the session.
`hooks`	A dictionary of hooks for the session.
`params`	A dictionary of parameters for the session.
`verify`	A boolean indicating whether to verify SSL certificates.
`cert`	A tuple of certificate files for the session.
`max_redirects`	An integer indicating the maximum number of redirects.
`trust_env`	A boolean indicating whether to trust the environment.

Session Methods

The `Session` class has the following methods:

Method	Description
`__init__`	Initializes a new session.
`prepare_request`	Prepares a request for sending.
`send`	Sends a request and returns a response.
`merge_environment_settings`	Merges environment settings into the session.
`get_adapter`	Returns an adapter for the session.
`close`	Closes the session.

SessionRedirectMixin Class

The `SessionRedirectMixin` class is a mixin class for session redirecting. It has the following methods:

Method	Description
`get_redirect_target`	Returns the redirect target for a response.
`is_redirect`	Checks if a response is a redirect.
`resolve_redirects`	Resolves redirects for a response.

merge_setting Function

The `merge_setting` function merges two settings dictionaries into one. It takes two dictionaries as arguments and returns a new dictionary containing the merged settings.

merge_hooks Function

The `merge_hooks` function merges two hooks dictionaries into one. It takes two dictionaries as arguments and returns a new dictionary containing the merged hooks.

session Function

The `session` function creates a new session. It takes no arguments and returns a new `Session` object.

Dependencies

The session management module depends on the following modules:

- `src/requests/adapters.py`
- `src/requests/auth.py`
- `src/requests/compat.py`
- `src/requests/cookies.py`

- src/requests/exceptions.py
- src/requests/hooks.py
- src/requests/models.py
- src/requests/status_codes.py
- src/requests/structures.py
- src/requests/utils.py
- src/requests/_internal_utils.py
- tests/compat.py
- tests/test_adapters.py
- tests/test_hooks.py
- tests/test_structures.py
- tests/test_utils.py
- tests/utils.py
- docs/dev/authors.rst
- docs/user/authentication.rst

CHAPTER 4: REQUEST AND RESPONSE MODELING

Overview

This chapter describes the modeling of requests and responses in the system.

Request Model

The request model is defined in `src/requests/models.py`. The following table describes the symbols defined in this module:

Symbol	Description
`RequestEncodingMixin`	Mixin class for encoding requests
`RequestHooksMixin`	Mixin class for adding hooks to requests
`Request`	Class representing a request
`PreparedRequest`	Class representing a prepared request
`Response`	Class representing a response

The `Request` class has the following attributes:

Attribute	Description
`method`	The HTTP method of the request (e.g. GET, POST, etc.)
`url`	The URL of the request
`headers`	The headers of the request
`data`	The data of the request
`params`	The query parameters of the request

The `PreparedRequest` class has the following attributes:

Attribute	Description
`method`	The HTTP method of the request (e.g. GET, POST, etc.)
`url`	The URL of the request
`headers`	The headers of the request

Attribute	Description
`data`	The data of the request
`params`	The query parameters of the request

The `Response` class has the following attributes:

Attribute	Description
`status_code`	The HTTP status code of the response
`headers`	The headers of the response
`content`	The content of the response

Request and Response Cycle

The following sequence diagram illustrates the request and response cycle:

1. The client sends a request to the server.
2. The server receives the request and processes it.
3. The server sends a response back to the client.
4. The client receives the response and processes it.

Request and Response Testing

The following tests are defined in `tests/test_requests.py`:

Test	Description
`test_json_encodes_as_bytes`	Test that JSON data is encoded as bytes
`test_requests_are_updated_each_time`	Test that requests are updated each time they are sent
`test_proxy_env_vars_override_default`	Test that proxy environment variables override default settings
`test_data_argument_accepts_tuples`	Test that the data argument accepts tuples
`test_prepared_copy`	Test that prepared requests can be copied
`test_urllib3_retries`	Test that urllib3 retries are handled correctly
`test_urllib3_pool_connection_closed`	Test that urllib3 pool connections are closed correctly

Request and Response Dependencies

The following dependencies are required for the request and response model:

Dependency	Description
src/requests/auth.py	Authentication module
src/requests/compat.py	Compatibility module
src/requests/cookies.py	Cookies module
src/requests/exceptions.py	Exceptions module
src/requests/hooks.py	Hooks module
src/requests/sessions.py	Sessions module
src/requests/status_codes.py	Status codes module
src/requests/structures.py	Structures module
src/requests/utils.py	Utilities module
src/requests/_internal_utils.py	Internal utilities module
src/requests/__version__.py	Version module

Request and Response Code Structure

The code for the request and response model is structured as follows:

- src/requests/models.py: Request and response model definitions
- tests/test_requests.py: Request and response tests
- src/requests/auth.py: Authentication module
- src/requests/compat.py: Compatibility module
- src/requests/cookies.py: Cookies module
- src/requests/exceptions.py: Exceptions module
- src/requests/hooks.py: Hooks module
- src/requests/sessions.py: Sessions module
- src/requests/status_codes.py: Status codes module
- src/requests/structures.py: Structures module
- src/requests/utils.py: Utilities module
- src/requests/_internal_utils.py: Internal utilities module
- src/requests/__version__.py: Version module

CHAPTER 5: ADAPTERS AND TRANSPORT

Overview

The Adapters and Transport layer is responsible for managing the connection between the client and the server. This layer provides a standardized interface for sending and receiving data over various transport protocols, such as HTTP.

Symbols

Symbol	Description	Location
`_urllib3_request_context`	Internal context for urllib3 request handling	`src/requests/adapters.py`
`BaseAdapter`	Base class for all adapters	`src/requests/adapters.py`
`HTTPAdapter`	Adapter for HTTP protocol	`src/requests/adapters.py`

Dependencies

Dependency	Description	Location
`src/requests/auth.py`	Authentication module	`src/requests/auth.py`
`src/requests/compat.py`	Compatibility module	`src/requests/compat.py`
`src/requests/cookies.py`	Cookies module	`src/requests/cookies.py`
`src/requests/exceptions.py`	Exceptions module	`src/requests/exceptions.py`
`src/requests/models.py`	Models module	`src/requests/models.py`
`src/requests/structures.py`	Structures module	`src/requests/structures.py`
`src/requests/utils.py`	Utilities module	`src/requests/utils.py`
`src/requests/_internal_utils.py`	Internal utilities module	`src/requests/_internal_utils.py`
`tests/compat.py`	Compatibility tests	`tests/compat.py`
`tests/test_structures.py`	Structures tests	`tests/test_structures.py`
`tests/test_utils.py`	Utilities tests	`tests/test_utils.py`
`tests/utils.py`	Test utilities	`tests/utils.py`

Dependency	Description	Location
`docs/dev/authors.rst`	Authors documentation	`docs/dev/authors.rst`
`docs/user/authentication.rst`	Authentication documentation	`docs/user/authentication.rst`

Connection Management

The Adapters and Transport layer manages the connection between the client and the server. This includes establishing and closing connections, as well as handling connection errors.

Request Handling

The Adapters and Transport layer handles requests from the client and sends them to the server. This includes formatting the request, sending the request, and handling the response.

Transport Protocols

The Adapters and Transport layer supports multiple transport protocols, including HTTP.

Test Cases

The Adapters and Transport layer has several test cases to ensure its functionality. These test cases cover various scenarios, including:

- `test_request_url_trims_leading_path_separators`: Test that the request URL trims leading path separators.

Code Organization

The code for the Adapters and Transport layer is organized into several files:

- `src/requests/adapters.py`: Contains the adapter classes and internal context for urllib3 request handling.
- `tests/test_adapters.py`: Contains test cases for the Adapters and Transport layer.